2026

Elaborato 1

ELABORATO_DEL_2025_11_19_12_06_18 ESAME DEL 27/11/2026

MASSIMILIANO SESSA

Sommario

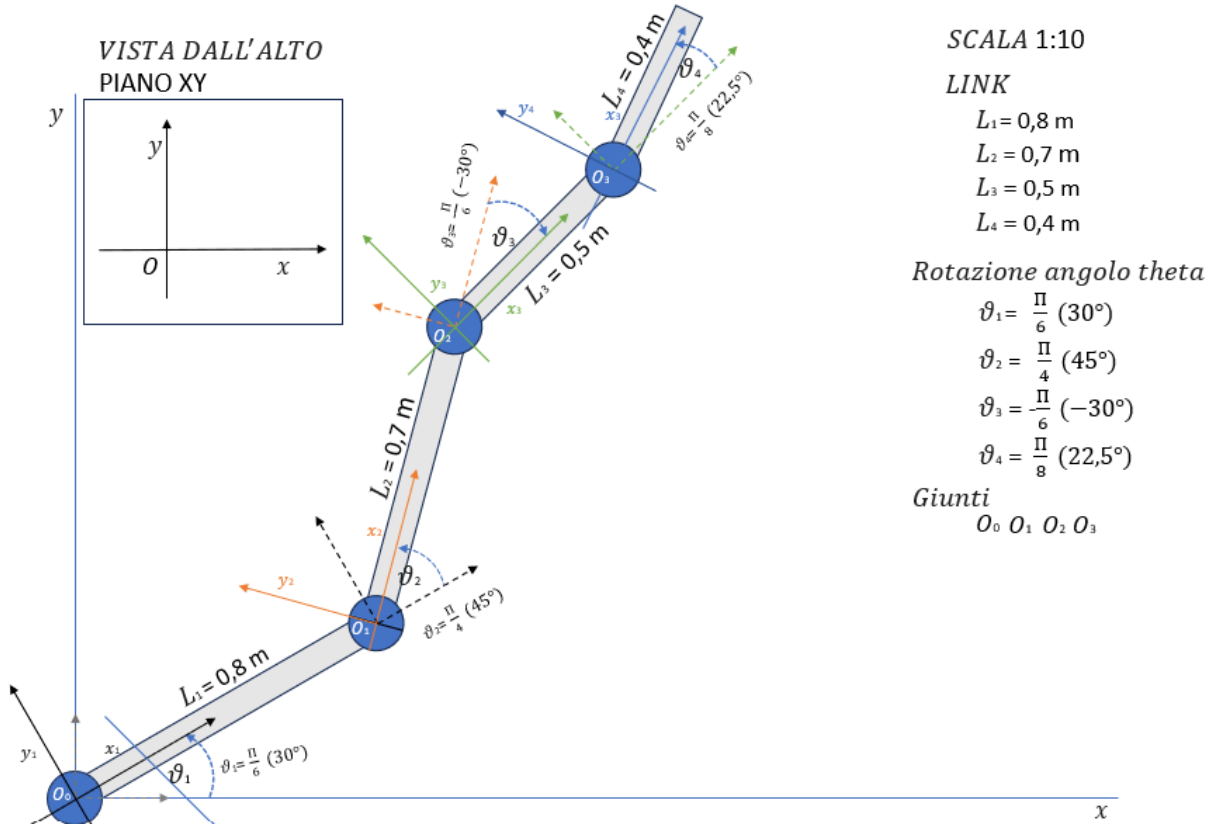
Cinematica diretta 2
Trasformazione Complessiva e Posizione/Orientamento 4
Cinematica indiretta 12
DINAMICA CON EULERO-LAGRANGE 13
CONTROLLORE 15
VISUALIZZAZIONE E ANIMAZIONE 15
File del progetto: 15
Output di matlab 16
Git 17

Cinematica diretta

Assegnazione delle Terne

Definisco un robot manipolatore planare composto da 4 giunti rotoidali (4R) con assi paralleli all'asse $\boldsymbol{z}$ ed ortogonali al piano di lavoro $\boldsymbol{xy}$.

- L'origine del sistema O è posta alla base del robot.
- Gli assi $\boldsymbol{z_1, z_2, z_3, z_4}$ sono tutti paralleli tra loro e uscenti dal piano di lavoro
- L'asse $\boldsymbol{z}$ è per perpendicolare al piano $\boldsymbol{xy}$
- L'asse $\boldsymbol{x}$ di ogni terna è allineato lungo il rispettivo link $\mathbf{L}$ in posizione di zero



Realizzazione plottata con MatLab

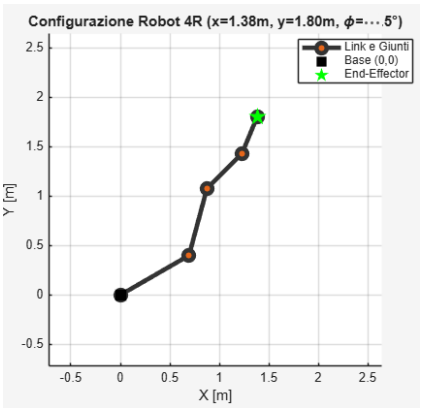


Tabella dei Parametri Denavit-Hartenberg (DH)

In accordo con le terne di riferimento definisco la tabella di Denavit – Hartenberg per il manipolatore planare 4R. Essendo tutti i link vincolati a muoversi nel piano $\boldsymbol{xy}$ e tutti gli assi di rotazione $\boldsymbol{z_i}$ paralleli tra loro nonché ortogonali al piano di lavoro, i parametri di offset $\boldsymbol{d_i}$ e di torsione $\boldsymbol{\alpha_i}$ risultano identicamente nulli per ogni giunto. Le lunghezze dei link $\boldsymbol{L_i}$ corrispondono ai parametri $\boldsymbol{a_i}$, mentre le variabili di giunto sono gli angoli $\boldsymbol{\vartheta_i}$

Giunto i	$\boldsymbol{\vartheta_i}$**(var angoli)**	$\boldsymbol{d_i}$**(torsione)**	$\boldsymbol{a_i}$ **(lunghezza link)**	$\boldsymbol{\alpha_i}$**(torsione)**
1	ϑ_1	0	L_1	0
2	ϑ_2	0	L_2	0
3	ϑ_3	0	L_3	0
4	ϑ_4	0	L_4	0

Matrici di Trasformazione Elementari A_i

Nella convenzione Denavit-Hartenberg standard, la matrice di trasformazione omogenea A_i che esprime la posa della terna i rispetto alla terna $i-1$ è definita dalla composizione di quattro trasformazioni elementari:

$$A_i = Rot_z(\vartheta_i)\, Trans_z(d_i)\, Trans_x(a_i)\, Rot_x(\alpha_i) =$$

$$\begin{bmatrix} \cos\vartheta_i & -sen\,\vartheta_i \cos\alpha_i & sen\,\vartheta_i \cos\alpha_i & a_i \cos\vartheta_i \\ sen\,\vartheta_i & \cos\vartheta_i \cos\alpha_i & -\cos\vartheta_i \cos\alpha_i & a_i\, sen\,\vartheta_i \\ 0 & sen\,\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Sostituisco i parametri specifici del manipolatore planare $d_i = 0$, $\alpha_i = 0$ e $a_i = L_i$ semplificando le matrici ai 4 giunti A_i dove $i = 1-4$

$$A_1 \begin{bmatrix} c_1 & -s_1 & 0 & L_1 c_1 \\ s_1 & c_1 & 0 & L_1 s_1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad A_2 \begin{bmatrix} c_2 & -s_2 & 0 & L_2 c_2 \\ s_2 & c_2 & 0 & L_2 s_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_3 \begin{bmatrix} c_3 & -s_3 & 0 & L_3 c_3 \\ s_3 & c_3 & 0 & L_3 s_3 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad A_4 \begin{bmatrix} c_4 & -s_4 & 0 & L_4 c_4 \\ s_4 & c_4 & 0 & L_4 s_4 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Dove:

$c_i = \cos\vartheta_i$

$s_i = sen\,\vartheta_i$

Trasformazione Complessiva e Posizione/Orientamento

La trasformazione complessiva descrive la posizione all'end effector partendo dalla terna base e si ottiene moltiplicando in sequenza le matrici:

$$\boldsymbol{T = A_1 A_2 A_3 A_4 \begin{bmatrix} c_{1234} & -s_{1234} & 0 & x \\ s_{1234} & c_{1234} & 0 & y \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}}$$

Dove:

$\Phi = \vartheta_1 + \vartheta_2 + \vartheta_3 + \vartheta_4$ dunque cosΦ e s_{1234}= sinΦ

$$x = L_1 cos\,\vartheta_1 + L_2 cos(\vartheta_1 + \vartheta_2) + L_3 cos(\vartheta_1 + \vartheta_2 + \vartheta_3) + L_4 cos(\vartheta_1 + \vartheta_2 + \vartheta_3 + \vartheta_4)$$

$$y = L_1 sen\,\vartheta_1 + L_2 sen(\vartheta_1 + \vartheta_2) + L_3 sen(\vartheta_1 + \vartheta_2 + \vartheta_3) + L_4 sen(\vartheta_1 + \vartheta_2 + \vartheta_3 + \vartheta_4)$$

Dall'ultima colonna della matrice $\boldsymbol{T}$ estraggo le coordinate dell'end effector e successivamente ricavo l'orientamento complessivo Φ dell'end effector estraendo $T_{1,1}$ e $T_{2,1}$ che per un manipolatore planare R4 coincide con la somma algebrica degli angoli dei giunti ovvero:

$$\Phi = \vartheta_1 + \vartheta_2 + \vartheta_3 + \vartheta_4$$

Realizzazione dei file matlab:

movimentazione_braccio.m:

```
% Terne e Parametri di Denavit - Hartenberg
clear; clc;

% qua definisco numericamente la matrice dei link
L = [0.8, 0.7, 0.5, 0.4];          %Li (i=1-4)
theta = [pi/6, pi/4, -pi/6, pi/8]; % Angoli di giunto thetha i (i=1-4)

% Matrice DH standard: ogni riga è [theta_i, d_i, a_i, alpha_i]
DH_num = [
    theta(1),  0,  L(1),  0;
    theta(2),  0,  L(2),  0;
    theta(3),  0,  L(3),  0;
    theta(4),  0,  L(4),  0
    ];

disp('Tabella DH numerica [theta, d, a, alpha]:');
disp(DH_num);

%  Definizione Simbolica
syms th1 th2 th3 th4 L1 L2 L3 L4 real

DH_sym = [
    th1,  0,  L1,  0;
    th2,  0,  L2,  0;
    th3,  0,  L3,  0;
    th4,  0,  L4,  0
    ];
```

```
disp('Tabella DH simbolica [theta, d, a, alpha]:');
disp(DH_sym);




% costruisco e calcolo le matrici di trasformazione A_i
% Funzione anonima per la matrice di trasformazione DH standard
dh_mat = @(th, d, a, al) [
    cos(th), -sin(th)*cos(al),  sin(th)*sin(al), a*cos(th);
    sin(th),  cos(th)*cos(al), -cos(th)*sin(al), a*sin(th);
    0,        sin(al),          cos(al),         d;
    0,        0,                0,               1
    ];

% Lavoro le matrici A_i
% Calcolo Numerico con un unico ciclo e lo stampo a display
A_num = cell(1, 4);
for i = 1:4
    A_num{i} = dh_mat(DH_num(i,1), DH_num(i,2), DH_num(i,3), DH_num(i,4));
    fprintf('Matrice numerica A_%d:\n', i);
    disp(A_num{i});
end

% Calcolo Simbolico con un unico ciclo e lo stampo a display
A_sym = cell(1, 4);
for i = 1:4
    A_sym{i} = dh_mat(DH_sym(i,1), DH_sym(i,2), DH_sym(i,3), DH_sym(i,4));
    fprintf('Matrice simbolica A_%d:\n', i);
    disp(A_sym{i});
end

% TRASFORMAZIONE COMPLESSIVA T ED ESTRAZIONE (x, y, phi)

% --- A. Calcolo Simbolico ---
T_sym = simplify(A_sym{1} * A_sym{2} * A_sym{3} * A_sym{4});

disp('Matrice di trasformazione complessiva simbolica T:');
disp(T_sym);

% Estrazione simbolica esplicita
x_sym   = T_sym(1, 4);
y_sym   = T_sym(2, 4);
phi_sym = th1 + th2 + th3 + th4; % Equivale a atan2(T_sym(2,1), T_sym(1,1))

fprintf('x(theta) = %s\n', char(x_sym));
fprintf('y(theta) = %s\n', char(y_sym));
fprintf('phi(theta) = %s\n\n', char(phi_sym));

% Calcolo Numerico
T_num = A_num{1} * A_num{2} * A_num{3} * A_num{4};

disp('Matrice di trasformazione complessiva numerica T:');
disp(T_num);

% Estrazione numerica esplicita
x_num   = T_num(1, 4);
y_num   = T_num(2, 4);
phi_num = atan2(T_num(2, 1), T_num(1, 1)); % Orientamento in radianti [-pi, pi]
```

```
fprintf('Posizione End-Effector: x = %.4f m, y = %.4f m\n', x_num, y_num);
fprintf('Orientamento End-Effector: phi = %.4f rad (%.2f deg)\n', phi_num,
rad2deg(phi_num));

% FUNZIONE fk_4R E GRAFICO DELLA CONFIGURAZIONE

% Richiamo della funzione con i parametri definiti sopra
[T_fun, x_fun, y_fun, phi_fun] = fun_4R(theta, L);

fprintf('Verifico la funzione fun_4R:\n');
fprintf('x = %.4f m, y = %.4f m, phi = %.4f rad (%.2f deg)\n', ...
    x_fun, y_fun, phi_fun, rad2deg(phi_fun));

% Calcolo delle posizioni dei giunti del grafico

x_joints = zeros(1, 5);
y_joints = zeros(1, 5);
ang_comp = 0;

for i = 1:4
    ang_comp = ang_comp + theta(i);
    x_joints(i+1) = x_joints(i) + L(i)*cos(ang_comp);
    y_joints(i+1) = y_joints(i) + L(i)*sin(ang_comp);
end

% Generazione del grafico
figure('Name', 'Cinematica Diretta - del braccio planare4R');
hold on; grid on; axis equal;

% Disegno dei link e dei giunti
plot(x_joints, y_joints, '-o', 'Color', [0.2 0.2 0.2], 'LineWidth', 3.5, ...
    'MarkerFaceColor', [0.9 0.4 0.1], 'MarkerSize', 8, 'DisplayName', 'Link e Giunti');

% Evidenzio la base (Giunto 1) e l'End-Effector
plot(0, 0, 'ks', 'MarkerSize', 10, 'MarkerFaceColor', 'k', 'DisplayName', 'Base
(0,0)');
plot(x_fun, y_fun, 'gp', 'MarkerSize', 12, 'MarkerFaceColor', 'g', 'DisplayName', 'End-
Effector');

% Etichette e limiti degli assi
L_tot = sum(L);
xlim([-L_tot*0.3, L_tot*1.1]);
ylim([-L_tot*0.3, L_tot*1.1]);
xlabel('X [m]');
ylabel('Y [m]');
title(sprintf('Configurazione Robot 4R (x=%.2fm, y=%.2fm, \\phi=%.1f°)', ...
    x_fun, y_fun, rad2deg(phi_fun)));
legend('Location', 'best');


% CINEMATICA INVERSA

% Punto target desiderato p = (x_des, y_des, phi_des)
x_des   = 1.2;
y_des   = 0.8;
phi_des = pi/4;

% Calcolo delle due soluzioni multiple
th_elbow_down = ifun_4R(x_des, y_des, phi_des, L,  1); % Gomito basso (+1)
th_elbow_up   = ifun_4R(x_des, y_des, phi_des, L, -1); % Gomito alto (-1)
```

```
[~, x_ver_d, y_ver_d, phi_ver_d] = fun_4R(th_elbow_down, L);
[~, x_ver_u, y_ver_u, phi_ver_u] = fun_4R(th_elbow_up, L);

fprintf('--- VERIFICA CINEMATICA INVERSA ---\n');
fprintf('Target:       x = %.4f, y = %.4f, phi = %.4f rad\n', x_des, y_des, phi_des);
fprintf('Gomito Basso: x = %.4f, y = %.4f, phi = %.4f rad (Errore pos: %.2e)\n', ...
    x_ver_d, y_ver_d, phi_ver_d, norm([x_des - x_ver_d, y_des - y_ver_d]));
fprintf('Gomito Alto:  x = %.4f, y = %.4f, phi = %.4f rad (Errore pos: %.2e)\n', ...
    x_ver_u, y_ver_u, phi_ver_u, norm([x_des - x_ver_u, y_des - y_ver_u]));

% Grafico comparativo delle due soluzioni
figure('Name', 'Cinematica Inversa - Soluzioni Multiple');
hold on; grid on; axis equal;

% Funzione helper inline per estrarre i punti dei giunti
get_pts = @(th) [0, cumsum(L .* cos(cumsum(th))); 0, cumsum(L .* sin(cumsum(th)))];

pts_d = get_pts(th_elbow_down);
pts_u = get_pts(th_elbow_up);

plot(pts_d(1,:), pts_d(2,:), '-o', 'LineWidth', 3, 'Color', [0 0.45 0.74],
'DisplayName', 'Gomito Basso');
plot(pts_u(1,:), pts_u(2,:), '--s', 'LineWidth', 3, 'Color', [0.85 0.33 0.1],
'DisplayName', 'Gomito Alto');
plot(x_des, y_des, 'gp', 'MarkerSize', 14, 'MarkerFaceColor', 'g', 'DisplayName',
'Target');

xlim([-sum(L)*0.3, sum(L)*1.1]); ylim([-sum(L)*0.5, sum(L)*1.1]);
xlabel('X [m]'); ylabel('Y [m]');
title('Verifica IK: Soluzioni Multiple (Gomito Alto vs Gomito Basso)');
legend('Location', 'best');




% SOTTO LA DINAMICA SIMBOLICA CON EULERO-LAGRANGE
syms th1 th2 th3 th4 dth1 dth2 dth3 dth4 real
syms m1 m2 m3 m4 L1 L2 L3 L4 lc1 lc2 lc3 lc4 I1 I2 I3 I4 g real

q_sym  = [th1; th2; th3; th4];
dq_sym = [dth1; dth2; dth3; dth4];

% i baricentri
xc1 = lc1*cos(th1); yc1 = lc1*sin(th1);
xc2 = L1*cos(th1) + lc2*cos(th1+th2); yc2 = L1*sin(th1) + lc2*sin(th1+th2);
xc3 = L1*cos(th1) + L2*cos(th1+th2) + lc3*cos(th1+th2+th3);
yc3 = L1*sin(th1) + L2*sin(th1+th2) + lc3*sin(th1+th2+th3);
xc4 = L1*cos(th1) + L2*cos(th1+th2) + L3*cos(th1+th2+th3) + lc4*cos(th1+th2+th3+th4);
yc4 = L1*sin(th1) + L2*sin(th1+th2) + L3*sin(th1+th2+th3) + lc4*sin(th1+th2+th3+th4);

% le velocità lineari e angolari
v1_sq = diff(xc1,th1)^2*dth1^2 + diff(yc1,th1)^2*dth1^2; % Semplificato via Jacobiano
J1 = jacobian([xc1; yc1], q_sym); J2 = jacobian([xc2; yc2], q_sym);
J3 = jacobian([xc3; yc3], q_sym); J4 = jacobian([xc4; yc4], q_sym);

T_kin = 0.5*(m1*(dq_sym'*(J1'*J1)*dq_sym) + I1*dth1^2 + ...
    m2*(dq_sym'*(J2'*J2)*dq_sym) + I2*(dth1+dth2)^2 + ...
```

```
    m3*(dq_sym'*(J3'*J3)*dq_sym) + I3*(dth1+dth2+dth3)^2 + ...
    m4*(dq_sym'*(J4'*J4)*dq_sym) + I4*(dth1+dth2+dth3+dth4)^2);

V_pot = g * (m1*yc1 + m2*yc2 + m3*yc3 + m4*yc4);

% Estrazione matrici M(q), G(q), C(q, dq)
M_sym = simplify(hessian(T_kin, dq_sym));
G_sym = simplify(jacobian(V_pot, q_sym)');

C_sym = sym(zeros(4,4));
for k = 1:4
    for j = 1:4
        for i = 1:4
            c_ijk = 0.5 * (diff(M_sym(k,j), q_sym(i)) + diff(M_sym(k,i), q_sym(j)) -
diff(M_sym(i,j), q_sym(k)));
            C_sym(k,j) = C_sym(k,j) + c_ijk * dq_sym(i);
        end
    end
end
C_sym = simplify(C_sym);
disp('Matrici simboliche M_sym, C_sym, G_sym calcolate.');




% CONTROLLORE, SIMULAZIONE, GRAFICI E ANIMAZIONE

% Parametri fisici del manipolatore
params.L  = L;
params.lc = L / 2;
params.m  = [2.0, 1.5, 1.0, 0.5];
params.I  = (1/12) * params.m .* (L.^2);
params.g  = 9.81;

% Parametri traiettoria circolare desiderata
t_span = [0 6];
xc = 1.2; yc = 0.5; R_traj = 0.35;
omega = 2*pi / 6;                       % 1 giro completo in 6 s
phi_d = 0;                              % Orientamento costante

% Condizioni iniziali (partenza sulla traiettoria a t = 0)
q0  = ifun_4R(xc + R_traj, yc, phi_d, params.L, 1)';
dq0 = [0; 0; 0; 0];

% Integrazione dinamica con ode45
opts = odeset('RelTol', 1e-4, 'AbsTol', 1e-6);
[t_sim, state] = ode45(@(t, s) robot_ode(t, s, params, xc, yc, R_traj, omega, phi_d),
...
                       t_span, [q0; dq0], opts);

q_sim  = state(:, 1:4);
dq_sim = state(:, 5:8);

% Ricostruzione di errore e(t), coppie tau(t) e traiettoria
N = length(t_sim);
e_sim   = zeros(N, 4);
tau_sim = zeros(N, 4);
xy_sim  = zeros(N, 2);
xy_des  = zeros(N, 2);
```

```
for k = 1:N
    [qd, dqd, ddqd, xd, yd] = get_traj(t_sim(k), params.L, xc, yc, R_traj, omega,
phi_d);
    xy_des(k, :) = [xd, yd];

    [~, x_c, y_c, ~] = fun_4R(q_sim(k, :)', params.L);
    xy_sim(k, :) = [x_c, y_c];

    e_sim(k, :)   = (qd - q_sim(k, :)')';
    tau_sim(k, :) = compute_tau(q_sim(k, :)', dq_sim(k, :)', qd, dqd, ddqd, params)';
end

% Grafici: q(t), e(t), tau(t)
figure('Name', 'Risultati Controllo 4R', 'Position', [100, 100, 900, 650]);

subplot(3,1,1);
plot(t_sim, rad2deg(q_sim), 'LineWidth', 1.5); grid on;
ylabel('q(t) [deg]'); title('Andamento delle variabili di giunto q(t)');
legend('\theta_1', '\theta_2', '\theta_3', '\theta_4', 'Location', 'best');

subplot(3,1,2);
plot(t_sim, rad2deg(e_sim), 'LineWidth', 1.5); grid on;
ylabel('e(t) [deg]'); title('Errore di inseguimento e(t) = q_d - q');
legend('e_1', 'e_2', 'e_3', 'e_4', 'Location', 'best');

subplot(3,1,3);
plot(t_sim, tau_sim, 'LineWidth', 1.5); grid on;
xlabel('Tempo [s]'); ylabel('\tau(t) [Nm]'); title('Coppie di controllo \tau(t)');
legend('\tau_1', '\tau_2', '\tau_3', '\tau_4', 'Location', 'best');

% Animazione con terne DH, target e drawnow
figure('Name', 'Animazione Manipolatore 4R', 'Position', [150, 150, 700, 700]);
step = max(1, floor(N / 100));

for k = 1:step:N
    clf; hold on; grid on; axis equal;
    xlim([-0.5, 2.5]); ylim([-1.0, 2.0]);
    xlabel('X [m]'); ylabel('Y [m]');
    title(sprintf('Animazione 4R - t = %.2f s', t_sim(k)));

    % Target e scia End-Effector
    plot(xy_des(:,1), xy_des(:,2), 'g--', 'LineWidth', 1.5, 'DisplayName', 'Target');
    plot(xy_sim(1:k,1), xy_sim(1:k,2), 'b:', 'LineWidth', 1.5, 'DisplayName',
'Traiettoria EE');

    % Posizioni giunti
    q_k = q_sim(k, :);
    px = zeros(1, 5); py = zeros(1, 5);
    ang = 0;
    for i = 1:4
        ang = ang + q_k(i);
        px(i+1) = px(i) + params.L(i)*cos(ang);
        py(i+1) = py(i) + params.L(i)*sin(ang);
    end

    % Disegno robot
    plot(px, py, '-o', 'Color', [0.2 0.2 0.2], 'LineWidth', 3.5, ...
         'MarkerFaceColor', [0.9 0.4 0.1], 'MarkerSize', 7, 'DisplayName', 'Robot 4R');

    % Terne DH (X_i rosso, Y_i blu)
    sc = 0.2; ang_dh = 0;
```

```
    for i = 1:4
        ang_dh = ang_dh + q_k(i);
        quiver(px(i+1), py(i+1),  sc*cos(ang_dh), sc*sin(ang_dh), 0, 'r', 'LineWidth',
1.2);
        quiver(px(i+1), py(i+1), -sc*sin(ang_dh), sc*cos(ang_dh), 0, 'b', 'LineWidth',
1.2);
    end

    drawnow;
end

%  FUNZIONI LOCALI PER SIMULAZIONE
function ds = robot_ode(t, s, params, xc, yc, R, omega, phi_d)
    q  = s(1:4);
    dq = s(5:8);
    [qd, dqd, ddqd, ~, ~] = get_traj(t, params.L, xc, yc, R, omega, phi_d);
    tau = compute_tau(q, dq, qd, dqd, ddqd, params);
    [M, C, G] = dyn_4R(q, dq, params);
    ddq = M \ (tau - C*dq - G);
    ds = [dq; ddq];
end

function tau = compute_tau(q, dq, qd, dqd, ddqd, params)
    [M, C, G] = dyn_4R(q, dq, params);
    Kp = diag([150, 150, 100, 80]);
    Kd = diag([25, 25, 20, 15]);
    tau = M * (ddqd + Kd*(dqd - dq) + Kp*(qd - q)) + C*dq + G;
end

function [qd, dqd, ddqd, xd, yd] = get_traj(t, L, xc, yc, R, omega, phi_d)
    dt = 1e-4;
    xd = xc + R*cos(omega*t);
    yd = yc + R*sin(omega*t);
    qd   = ifun_4R(xd, yd, phi_d, L, 1)';
    qd_p = ifun_4R(xc + R*cos(omega*(t+dt)), yc + R*sin(omega*(t+dt)), phi_d, L, 1)';
    qd_m = ifun_4R(xc + R*cos(omega*(t-dt)), yc + R*sin(omega*(t-dt)), phi_d, L, 1)';
    dqd  = (qd_p - qd_m) / (2*dt);
    ddqd = (qd_p - 2*qd + qd_m) / (dt^2);
end
```

Realizzazione di **funzione fun_4R.m**

```
function [T, x, y, phi] = fun_4R(theta, L)
% Calcola della cinematica diretta per un manipolatore planare 4R

th1 = theta(1);
th2 = theta(2);
th3 = theta(3);
th4 = theta(4);

% Somme cumulative degli angoli
phi_1   = th1;
phi_12  = th1 + th2;
phi_123 = th1 + th2 + th3;
phi     = th1 + th2 + th3 + th4;

% Coordinate dell'End-Effector
x = L(1)*cos(phi_1) + L(2)*cos(phi_12) + L(3)*cos(phi_123) + L(4)*cos(phi);
```

```
y = L(1)*sin(phi_1) + L(2)*sin(phi_12) + L(3)*sin(phi_123) + L(4)*sin(phi);

% Matrice di trasformazione omogenea complessiva T
T = [cos(phi), -sin(phi), 0, x;
    sin(phi),  cos(phi), 0, y;
    0,         0,        1, 0;
    0,         0,        0, 1];
end
```

Con matlab calcolo :

```
Matrice di trasformazione complessiva numerica T:
    0.3827   -0.9239         0    1.3806
    0.9239    0.3827         0    1.7993
         0         0    1.0000         0
         0         0         0    1.0000

Posizione End-Effector: x = 1.3806 m, y = 1.7993 m
Orientamento End-Effector: phi = 1.1781 rad (67.50 deg)

Valori restituiti dalla funzione fun_4R
x = 1.3806 m, y = 1.7993 m, phi = 1.1781 rad (67.50 deg)
```

Cinematica indiretta

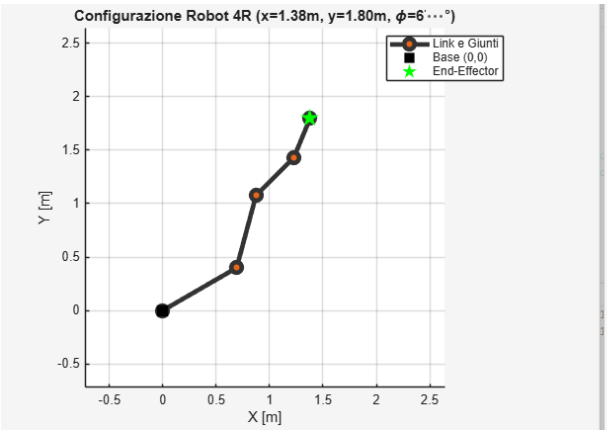
Equazione del "polso"

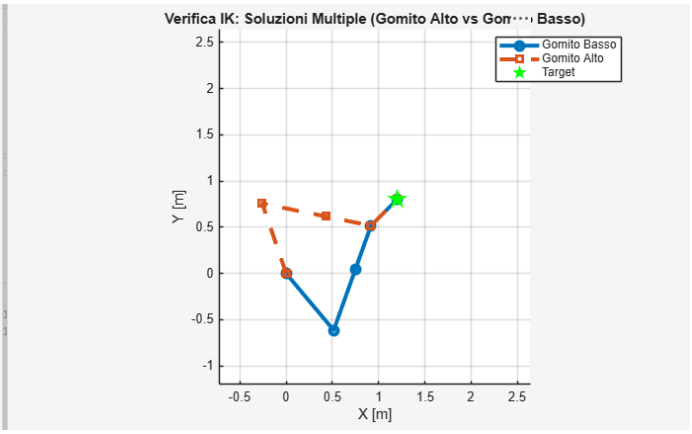
Procedo all'individuazione delle coordinate del polso (x_w, y_w)

$$x_w = x - L_4 \cos \Phi$$

$$y_w = y - L_4 \operatorname{sen} \Phi$$

Procedo al posizionamento e calcolo della posizione dei link considerando che per 3 link in un manipolatore planare 3R per raggiungere una posizione bidimensionale è presente un grado di ridondanza cinematica considero 2 e 3 collineare.





Realizzazione della funzione **ifun_4R.m** per cinematica inversa:

```
function theta = ifun_4R(x, y, phi, L, elbow)
% Calcola della cinematica inversa
%   elbow : +1 per gomito basso (default), -1 per gomito alto
% Output:

if nargin < 5
    elbow = 1; % posiziono il gomito basso
end

% Equazione del polso
xw = x - L(4)*cos(phi);
yw = y - L(4)*sin(phi);

% ricalcolo la posizione dei precedenti 3 link rendendo i link 2 e 3 collineari in
quanto un link risulta ridondante
L1  = L(1);
L23 = L(2) + L(3); % <--eccolo
r2  = xw^2 + yw^2;

D = (r2 - L1^2 - L23^2) / (2 * L1 * L23);

% Controllo raggiungibilità nello spazio di lavoro
if abs(D) > 1
    error('Punto (%.2f, %.2f) fuori dallo spazio di lavoro raggiungibile!', x, y);
end

% Calcolo theta2 (gomito alto / gomito basso)
th2 = atan2(elbow * sqrt(1 - D^2), D);

% Calcolo theta1
th1 = atan2(yw, xw) - atan2(L23 * sin(th2), L1 + L23 * cos(th2));

% Scelta di ridondanza per theta3
th3 = 0;

% 3. Calcolo del quarto giunto
th4 = phi - (th1 + th2 + th3);

theta = [th1, th2, th3, th4];
end
```

DINAMICA CON EULERO-LAGRANGE

Definito il vettore delle coordinate generalizzate q = [ϑ_1, ϑ_2, ϑ_3, ϑ_4]^T

Calcolo :

- **Energia Cinetica (T)**
- **Energia Potenziale (V)**
- **Equazioni del moto**

Predispongo la funzione dyn_4R per la simulazione di movimento

```
function [M, C, G] = dyn_4R(theta, dtheta, params)
  q = theta(:); dq = dtheta(:);
  M = calc_M(q, params);
  G = calc_G(q, params);

  % Matrice C tramite simboli di Christoffel numerici
  C = zeros(4, 4);
  eps_val = 1e-6;
  dM = zeros(4, 4, 4);
  for k = 1:4
    qp = q; qp(k) = qp(k) + eps_val;
    qm = q; qm(k) = qm(k) - eps_val;
    dM(:,:,k) = (calc_M(qp, params) - calc_M(qm, params)) / (2*eps_val);
  end
  for k = 1:4
    for j = 1:4
      for i = 1:4
        C(k,j) = C(k,j) + 0.5*(dM(k,j,i) + dM(k,i,j) - dM(i,j,k))*dq(i);
      end
    end
  end
end

function M = calc_M(q, p)
  M = zeros(4, 4);
  for i = 1:4
    Jv = zeros(2, 4); Jw = zeros(1, 4);
    for j = 1:i
      Jw(1, j) = 1;
      dx = 0; dy = 0;
      for k = j:i
        len = (k < i)*p.L(k) + (k == i)*p.lc(i);
        ang = sum(q(1:k));
        dx = dx - len*sin(ang);
        dy = dy + len*cos(ang);
      end
      Jv(1, j) = dx; Jv(2, j) = dy;
    end
    M = M + p.m(i)*(Jv'*Jv) + p.I(i)*(Jw'*Jw);
  end
end

function G = calc_G(q, p)
  G = zeros(4, 1);
  for j = 1:4
    for i = j:4
      dy = 0;
      for k = j:i
        len = (k < i)*p.L(k) + (k == i)*p.lc(i);
        dy = dy + len*cos(sum(q(1:k)));
      end
      G(j) = G(j) + p.m(i)*p.g*dy;
    end
```

```
    end
end
```

CONTROLLORE

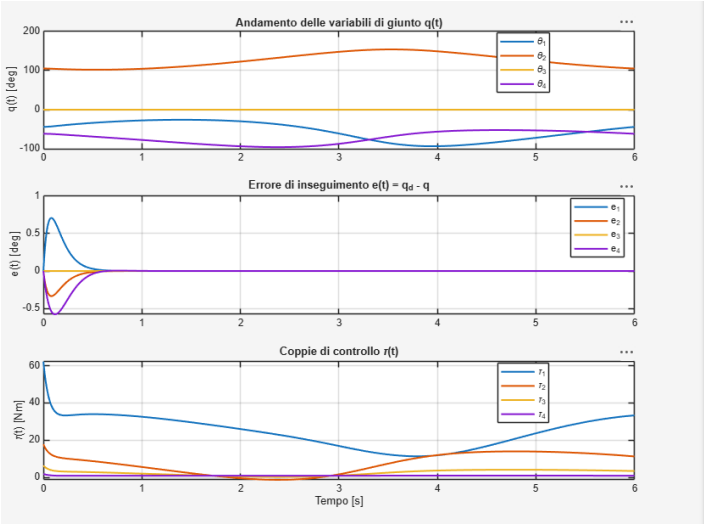
Il controllore è il **"pilota" del braccio robotico**. Il suo compito è tradurre una traiettoria ideale (dove vogliamo che vada il robot) in comandi fisici reali per i motori (quanta forza o coppia devono applicare i quattro giunti, istante per istante) affinché il movimento sia fluido, preciso e stabile.

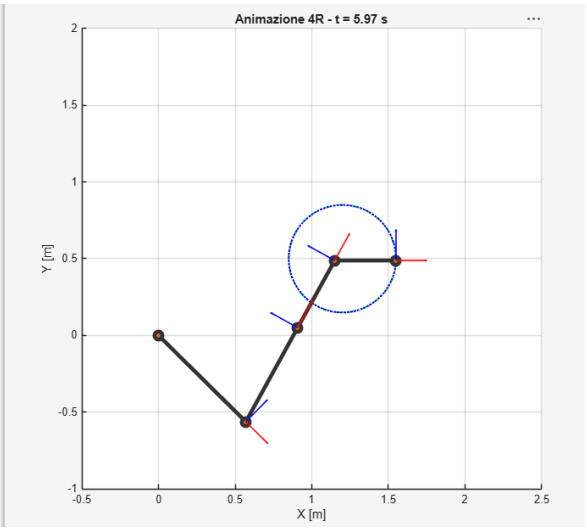
Un manipolatore è soggetto a disturbi fisici continui che ne deformano il comportamento come gravità, inerzia e accoppiamento dinamico pertanto il controllore serve a restituire veridicità alla simulazione calandola in un contesto verosimile.

Per il controllo di posizione lungo una traiettoria circolare nel piano xy si adotta un controllore a **coppia calcolata (*Computed Torque Control*)** nello spazio dei giunti:
Una traiettoria **polinomiale cubica o lineare a tratti** nello spazio dei giunti (o uno spostamento smooth da una configurazione iniziale a una finale).

VISUALIZZAZIONE E ANIMAZIONE

Procedo alla visualizzazione del movimento nonché dell'evoluzione temporale del manipolatore rappresentata graficamente tramite animazione con comando drawnow.
Questo comando si occupa di mostrare la configurazione ogni istante dei 4 link e le terne DH associate a ciascun giunto oltree la traiettoria di riferimento (target)





File del progetto (4):

- movimentazione_braccio.m → file principale
- fun_4R.m → funzioni di cinematica diretta
- ifun_4R.m → funzione di cinematica indiretta
- dyn_4R.m → simulazione e grafica

Output di matlab

```
Tabella DH numerica [theta, d, a, alpha]:
    0.5236         0    0.8000         0
    0.7854         0    0.7000         0
   -0.5236         0    0.5000         0
    0.3927         0    0.4000         0

Tabella DH simbolica [theta, d, a, alpha]:
[th1, 0, L1, 0]
[th2, 0, L2, 0]
[th3, 0, L3, 0]
[th4, 0, L4, 0]

Matrice numerica A_1:
    0.8660   -0.5000         0    0.6928
    0.5000    0.8660         0    0.4000
         0         0    1.0000         0
         0         0         0    1.0000

Matrice numerica A_2:
    0.7071   -0.7071         0    0.4950
    0.7071    0.7071         0    0.4950
         0         0    1.0000         0
         0         0         0    1.0000

Matrice numerica A_3:
    0.8660    0.5000         0    0.4330
   -0.5000    0.8660         0   -0.2500
         0         0    1.0000         0
         0         0         0    1.0000

Matrice numerica A_4:
    0.9239   -0.3827         0    0.3696
    0.3827    0.9239         0    0.1531
         0         0    1.0000         0
         0         0         0    1.0000

Matrice simbolica A_1:
[cos(th1), -sin(th1), 0, L1*cos(th1)]
[sin(th1),  cos(th1), 0, L1*sin(th1)]
[       0,         0, 1,           0]
[       0,         0, 0,           1]

Matrice simbolica A_2:
[cos(th2), -sin(th2), 0, L2*cos(th2)]
[sin(th2),  cos(th2), 0, L2*sin(th2)]
[       0,         0, 1,           0]
[       0,         0, 0,           1]

Matrice simbolica A_3:
[cos(th3), -sin(th3), 0, L3*cos(th3)]
[sin(th3),  cos(th3), 0, L3*sin(th3)]
[       0,         0, 1,           0]
[       0,         0, 0,           1]
```

```
Matrice simbolica A_4:
[cos(th4), -sin(th4), 0, L4*cos(th4)]
[sin(th4),  cos(th4), 0, L4*sin(th4)]
[       0,         0, 1,           0]
[       0,         0, 0,           1]
```

Matrice di trasformazione complessiva simbolica T:
[cos(th1 + th2 + th3 + th4), -sin(th1 + th2 + th3 + th4), 0, L4*cos(th1 + th2 + th3 + th4) + L2*cos(th1 + th2) + L1*cos(th1) + L3*cos(th1 + th2 + th3)]
[sin(th1 + th2 + th3 + th4), cos(th1 + th2 + th3 + th4), 0, L4*sin(th1 + th2 + th3 + th4) + L2*sin(th1 + th2) + L1*sin(th1) + L3*sin(th1 + th2 + th3)]
[0, 0, 1, 0]
[0, 0, 0, 1]

x(theta) = L4*cos(th1 + th2 + th3 + th4) + L2*cos(th1 + th2) + L1*cos(th1) + L3*cos(th1 + th2 + th3)
y(theta) = L4*sin(th1 + th2 + th3 + th4) + L2*sin(th1 + th2) + L1*sin(th1) + L3*sin(th1 + th2 + th3)
phi(theta) = th1 + th2 + th3 + th4

Matrice di trasformazione complessiva numerica T:
0.3827 -0.9239 0 1.3806
0.9239 0.3827 0 1.7993
0 0 1.0000 0
0 0 0 1.0000

Posizione End-Effector: x = 1.3806 m, y = 1.7993 m
Orientamento End-Effector: phi = 1.1781 rad (67.50 deg)
Verifico la funzione fun_4R:
x = 1.3806 m, y = 1.7993 m, phi = 1.1781 rad (67.50 deg)
--- VERIFICA CINEMATICA INVERSA ---
Target: x = 1.2000, y = 0.8000, phi = 0.7854 rad
Gomito Basso: x = 1.2000, y = 0.8000, phi = 0.7854 rad (Errore pos: 2.22e-16)
Gomito Alto: x = 1.2000, y = 0.8000, phi = 0.7854 rad (Errore pos: 2.48e-16)
Matrici simboliche M_sym, C_sym, G_sym calcolate.

Git

https://github.com/massimilianosessa/elaborato_robotica_1/

FINE